

LAB 6

- TASK 1

task1.cpp

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  using namespace std;
5  struct SensorReading {
6      int sensorID;
7      float temperature;
8      float distance;
9      string timestamp;
10 };
11 int main() {
12     ofstream outFile("sensor_log.txt");
13     SensorReading readings[3];
14     readings[0] = {101, 36.5, 120.3, "2025-06-01 08:00:00"};
15     readings[1] = {102, 42.1, 85.7, "2025-06-01 08:01:00"};
16     readings[2] = {103, 29.8, 200.0, "2025-06-01 08:02:00"};
17     for (int i = 0; i < 3; i++) {
18         outFile << readings[i].sensorID << endl;
19         outFile << readings[i].temperature << endl;
20         outFile << readings[i].distance << endl;
21         outFile << readings[i].timestamp << endl;
22         outFile << endl;
23     }
24     outFile.close();
25     cout << "Sensor data written to sensor_log.txt\n\n";
26     ifstream inFile("sensor_log.txt");
27     cout << "          ROBOT SENSOR LOG READINGS          \n";
28     SensorReading r;
29     string separator;
30     while (inFile >> r.sensorID >> r.temperature >> r.distance) {
31         inFile.ignore();
32         getline(inFile, r.timestamp);
33         getline(inFile, separator);
34         cout << "Sensor ID   : " << r.sensorID << endl;
35         cout << "Temperature : " << r.temperature << " C"<<endl;
36         cout << "Distance    : " << r.distance << " cm"<<endl;
37         cout << "Timestamp   : " << r.timestamp << endl;
38         cout << "-----" <<endl;
39     }
40     inFile.close();
41     return 0;
}
```

Sensor data written to sensor_log.txt

ROBOT SENSOR LOG READINGS

Sensor ID : 101
Temperature : 36.5 C
Distance : 120.3 cm
Timestamp : 2025-06-01 08:00:00

Sensor ID : 102
Temperature : 42.1 C
Distance : 85.7 cm
Timestamp : 2025-06-01 08:01:00

Sensor ID : 103
Temperature : 29.8 C
Distance : 200 cm
Timestamp : 2025-06-01 08:02:00

• TASK 2

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 using namespace std;
5 struct Student {
6     int rollNo;
7     string name;
8     string section;
9     float cgpa;
10 };
11 int main() {
12     ofstream outFile("student.txt");
13     Student students[2];
14     students[0] = {1, "Ali Hassan", "CS-A", 3.75};
15     students[1] = {2, "Sara Ahmed", "CS-B", 3.90};
16     for (int i = 0; i < 2; i++) {
17         outFile << students[i].rollNo << "\n";
18         outFile << students[i].name << "\n";
19         outFile << students[i].section << "\n";
20         outFile << students[i].cgpa << "\n";
21         outFile << "---\n";
22     }
23     outFile << "Total students: 2\n";
24     outFile.close();
25     cout << "Student records written to student.txt\n\n";
26     ifstream inFile("student.txt");
27     cout << "          STUDENT DATABASE RECORDS          \n";
28     cout << "===== \n";
29     Student s;
```

```

                        STUDENT DATABASE RECORDS
=====
Roll No   : 1
Name      : Ali Hassan
Section   : CS-A
CGPA      : 3.75
-----

Roll No   : 2
Name      : Sara Ahmed
Section   : CS-B
CGPA      : 3.9
-----

```

• TASK 3

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  using namespace std;
5  struct Account {
6      string name;
7      int accountNumber;
8      float* deposits;
9      float balance;
10 };
11 int main() {
12     const int NUM_ACCOUNTS = 5;
13     const int DEPOSIT_DAYS = 30;
14     Account accounts[NUM_ACCOUNTS];
15     accounts[0] = {"Usman ", 10001, new float[DEPOSIT_DAYS], 75000.0};
16     accounts[1] = {"Fatima ", 10002, new float[DEPOSIT_DAYS], 42000.0};
17     accounts[2] = {"Bilal ", 10003, new float[DEPOSIT_DAYS], 99000.0};
18     accounts[3] = {"Ayesha ", 10004, new float[DEPOSIT_DAYS], 15000.0};
19     accounts[4] = {"Zara ", 10005, new float[DEPOSIT_DAYS], 63000.0};
20     for (int i = 0; i < NUM_ACCOUNTS; i++) {
21         for (int d = 0; d < DEPOSIT_DAYS; d++) {
22             accounts[i].deposits[d] = 1000.0 + (i * 200) + (d * 10);
23         }
24     }
25     ofstream outFile("details.txt");
26     for (int i = 0; i < NUM_ACCOUNTS; i++) {
27         outFile << accounts[i].name << "\n";
28         outFile << accounts[i].accountNumber << "\n";
29         for (int d = 0; d < DEPOSIT_DAYS; d++) {
30             outFile << accounts[i].deposits[d];
31             if (d < DEPOSIT_DAYS - 1) outFile << " ";
32         }

```

```
Bank records written to details.txt

  ACCOUNTS WITH BALANCE > 50,000 PKR
=====
Name      : Usman
Acc No    : 10001
Balance   : PKR 75000
-----
Name      : Bilal
Acc No    : 10003
Balance   : PKR 99000
-----
Name      : Zara
Acc No    : 10005
Balance   : PKR 63000
-----
Memory freed successfully.
```

- HOME TASK 1

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <cmath>
5  using namespace std;
6  struct Waypoint {
7      int id;
8      float x, y, z;
9      string timestamp;
10 };
11 float euclideanDistance(Waypoint a, Waypoint b) {
12     float dx = b.x - a.x;
13     float dy = b.y - a.y;
14     float dz = b.z - a.z;
15     return sqrt(dx*dx + dy*dy + dz*dz);
16 }
17 int main() {
18     const int NUM_POINTS = 10;
19     Waypoint path[NUM_POINTS] = {
20         {1, 0.0, 0.0, 0.0, "08:00:00"},
21         {2, 10.0, 5.0, 1.0, "08:00:10"},
22         {3, 20.0, 10.0, 2.0, "08:00:20"},
23         {4, 35.0, 15.0, 3.0, "08:00:30"},
24         {5, 50.0, 20.0, 4.0, "08:00:40"},
25         {6, 70.0, 30.0, 5.0, "08:00:50"},
26         {7, 90.0, 40.0, 6.0, "08:01:00"},
27         {8, 105.0, 50.0, 7.0, "08:01:10"},
28         {9, 115.0, 60.0, 8.0, "08:01:20"},
29         {10, 80.0, 45.0, 5.0, "08:01:30"},
30     };
31     ofstream outFile("path_log.csv");
32     outFile << "waypoint_id,x,y,z,timestamp\n";

```

Path logged to path_log.csv

ROBOT PATH ANALYSIS

```
=====
WP 1 | X:0 Y:0 Z:0 @ 08:00:00
WP 2 | X:10 Y:5 Z:1 @ 08:00:10
WP 3 | X:20 Y:10 Z:2 @ 08:00:20
WP 4 | X:35 Y:15 Z:3 @ 08:00:30
WP 5 | X:50 Y:20 Z:4 @ 08:00:40
WP 6 | X:70 Y:30 Z:5 @ 08:00:50
WP 7 | X:90 Y:40 Z:6 @ 08:01:00
WP 8 | X:105 Y:50 Z:7 @ 08:01:10 <<< OUT OF SAFE ZONE
WP 9 | X:115 Y:60 Z:8 @ 08:01:20 <<< OUT OF SAFE ZONE
WP 10 | X:80 Y:45 Z:5 @ 08:01:30
=====
```

Total Distance : 169.332 units

Average Speed : 1.88146 units/sec

Flagged Points : 2 waypoints outside safe zone

Summary appended to path_log.csv

● HOME TASK 2

```

1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  struct Layer {
5      int neurons;
6      float weights[10];
7      float bias;
8  };
9  int main() {
10     const int NUM_LAYERS = 3;
11     Layer* network = new Layer[NUM_LAYERS];
12     network[0].neurons = 4;
13     network[0].bias = 0.1;
14     for (int w = 0; w < 10; w++)
15         network[0].weights[w] = 0.10 + w * 0.05;
16     network[1].neurons = 8;
17     network[1].bias = 0.25;
18     for (int w = 0; w < 10; w++)
19         network[1].weights[w] = 0.20 + w * 0.03;
20     network[2].neurons = 2;
21     network[2].bias = 0.5;
22     for (int w = 0; w < 10; w++)
23         network[2].weights[w] = 0.30 + w * 0.07;
24     ofstream outFile("model_weights.bin", ios::binary);
25     outFile.write((char*)network, sizeof(Layer) * NUM_LAYERS);
26     outFile.close();
27     cout << "Model weights saved to model_weights.bin\n\n";
28     delete[] network;
29     Layer* loaded = new Layer[NUM_LAYERS];
30     ifstream inFile("model_weights.bin", ios::binary);
31     inFile.read((char*)loaded, sizeof(Layer) * NUM_LAYERS);
32     inFile.close();

```

```
Model weights saved to model_weights.bin
```

```
NEURAL NETWORK WEIGHT VERIFY
```

```
=====
```

```
[Input Layer]
```

```
Neurons : 4
```

```
Bias    : 0.1
```

```
Weights : 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55
```

```
[Hidden Layer]
```

```
Neurons : 8
```

```
Bias    : 0.25
```

```
Weights : 0.2, 0.23, 0.26, 0.29, 0.32, 0.35, 0.38, 0.41, 0.44, 0.47
```

```
[Output Layer]
```

```
Neurons : 2
```

```
Bias    : 0.5
```

```
Weights : 0.3, 0.37, 0.44, 0.51, 0.58, 0.65, 0.72, 0.79, 0.86, 0.93
```

```
-----  
Binary read/write verified successfully!
```

- HOME TASK 3


```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  using namespace std;
5  struct SensorEntry {
6      int robotID;
7      int readingNum;
8      float temperature;
9      float battery;
10 };
11 void writeRobotLog(string filename, int robotID, float temps[], float batteries[]) {
12     ofstream f(filename.c_str());
13     f << robotID << "\n";
14     for (int i = 0; i < 5; i++) {
15         f << (i + 1) << " " << temps[i] << " " << batteries[i] << "\n";
16     }
17     f.close();
18 }
19 void readAndStore(string filename, SensorEntry entries[], int &count) {
20     ifstream f(filename.c_str());
21     if (!f.is_open()) {
22         cout << "Cannot open " << filename << "\n";
23         return;
24     }
25     int robotID;
26     f >> robotID;
27     for (int i = 0; i < 5; i++) {
28         SensorEntry e;
29         e.robotID = robotID;
30         f >> e.readingNum >> e.temperature >> e.battery;
31         entries[count++] = e;

```

Reading 1		Temp: 36.1C		Battery: 85%
Reading 2		Temp: 37.2C		Battery: 80%
Reading 3		Temp: 38C		Battery: 75%
Reading 4		Temp: 36.8C		Battery: 70%
Reading 5		Temp: 37.5C		Battery: 65%

[Robot 2]

Reading 1		Temp: 40C		Battery: 50%
Reading 2		Temp: 41.5C		Battery: 40%
Reading 3		Temp: 39.8C		Battery: 30%
Reading 4		Temp: 42C		Battery: 18% <<< CRITICAL - LOW BATTERY
Reading 5		Temp: 40.5C		Battery: 12% <<< CRITICAL - LOW BATTERY

[Robot 3]

Reading 1		Temp: 33C		Battery: 95%
Reading 2		Temp: 34.5C		Battery: 92%
Reading 3		Temp: 35C		Battery: 88%
Reading 4		Temp: 33.8C		Battery: 85%
Reading 5		Temp: 34.2C		Battery: 83%

=====

END OF FLEET LOG

=====